

Intern Assignment: Shopping Cart Application

Objective

Build a responsive shopping cart application where users can browse products, search and filter products, manage their cart, and complete a simple checkout flow.

This assignment is intended to evaluate your understanding of React, TypeScript, API integration, state management, validation, and frontend fundamentals.

Tech Stack

- React 19
- TypeScript
- Vite
- Tailwind CSS or Bootstrap
- pnpm
- Zustand
- TanStack Query
- Zod
- localStorage

Product API

Use a free public product API instead of local mock data.

Recommended API:

```
https://dummyjson.com/products
```

Use TanStack Query to fetch and manage product data.

Use Zod to validate the API response before using it in the application.

Core Features

Product Listing

Display at least 10 products in a responsive grid.

Each product should show:

- Product image
- Product title

- Category
- Price
- Rating
- Add to Cart button

The page should properly handle:

- Loading state
- API error state
- Empty state
- Successful response

Search and Filtering

Users should be able to:

- Search products by title
- Filter products by category
- Filter products by price
- Clear all filters

Use a custom hook to manage the search and filtering logic.

Cart Management

Users should be able to:

- Add products to the cart
- Remove products from the cart
- Increase product quantity
- Decrease product quantity
- Clear the cart

The minimum quantity for a cart item is .

The maximum quantity for a cart item is .

Use Zustand for global cart state management.

Cart Summary

Display the following information:

- Cart products
- Product quantity
- Individual product total
- Subtotal
- Tax

- Discount
- Final total

Calculation rules:

- Tax is 5% of the subtotal
- Apply a 10% discount when the subtotal is greater than \$100
- Minimum checkout value is \$10

Checkout should be disabled when the minimum cart value is not reached.

A clear message should explain why checkout is disabled.

Cart Persistence

Store cart data in `localStorage`.

The cart should remain available after refreshing the page.

You may use Zustand's persist middleware for this.

Checkout Steps

Create a simple checkout flow with the following steps:

1. Cart Review
2. Shipping
3. Payment Summary

The user should only move to the next step when the current step is valid.

Cart Review

Show the selected cart items and complete price summary.

Shipping

Create a basic shipping form containing:

- Full name
- Email
- Phone number
- Address
- City
- Postal code

Do not use form libraries such as React Hook Form or Formik.

Manage the form using React state.

Use Zod for form validation.

Show validation errors near the relevant fields.

Payment Summary

This step should be read-only.

Display:

- Shipping details
- Cart items
- Subtotal
- Tax
- Discount
- Final total

No payment gateway integration is required.

When the user clicks **Place Order**, display a success message and clear the cart.

Validation Requirements

Use Zod for:

- Product API response validation
- Shipping form validation

Shipping form validation should include:

- Full name is required
- Email must be valid
- Phone number is required
- Address is required
- City is required
- Postal code is required

State and Data Management

Use Zustand for cart state and cart actions.

Use TanStack Query only for API data fetching, caching, loading states, and error states.

Do not store fetched product data inside Zustand unless there is a clear reason to do so.

Use custom hooks where they improve readability and separate reusable logic.

UI and UX Requirements

- Responsive on mobile, tablet, and desktop
- Display a clear loading indicator
- Display a useful API error message
- Show an empty-cart message
- Show an empty-search-result message
- Disable quantity buttons when limits are reached
- Show the cart item count
- Show clear form validation messages
- Keep spacing, typography, and button styles consistent
- Avoid unnecessary animations and overcomplicated UI

Package Manager

Use `pnpm` to install and manage packages.

```
pnpm create vite shopping-cart --template react-ts
cd shopping-cart
pnpm install
pnpm add zustand @tanstack/react-query zod
```

Install either Tailwind CSS or Bootstrap for styling.

Deployment

Deploy the completed application using one of the following:

- Vercel
- Netlify
- Cloudflare Pages

The deployed application must be publicly accessible.

Submission Requirements

Submit:

- GitHub repository link
- Live deployment link
- README file

The README should contain:

- Project overview
- Technologies used
- Setup instructions
- Commands to run the project
- API used
- Features completed
- Known limitations

Evaluation Priority

The main priority is your understanding of the code you submit.

You should be able to explain:

- How product fetching works
- Why TanStack Query is used
- How the Zustand store works
- How cart persistence works
- How cart totals are calculated
- How Zod validation works
- How loading and error states are handled
- Why components and hooks were separated
- What improvements you would make with more time

A smaller and cleaner implementation that you fully understand is better than a large or complicated implementation that you cannot explain.

AI tools may be used for guidance, debugging, or learning. However, submitting randomly generated AI code without understanding, reviewing, and testing it will negatively affect the evaluation.

Code quality, reasoning, fundamentals, and the ability to explain implementation decisions will be prioritised over the number of features completed.

Evaluation Criteria

The assignment will be evaluated based on:

- Understanding of the submitted code
- React and TypeScript fundamentals
- Component design and readability
- Zustand state management
- TanStack Query usage
- Zod validation
- Correct cart calculations
- localStorage persistence

- Responsive design
- Error handling and user feedback
- Git commit quality
- Successful deployment

Optional Bonus Features

These are optional and should only be attempted after completing the core requirements:

- Product sorting
- Skeleton loading
- Dark mode
- Product details view
- Basic tests for cart calculation functions